

企业级应用软件设计与开发

期末大作业报告

DataAgent Java AI 智能数据库查询助手

方向一：Agentic AI 原生开发

学	号	2025302961
姓	名	曹腾飞
专	业	计算机技术
指	导	戚欣

提交日期：2026 年 6 月 22 日

目录

一、选题背景与设计思想

- [1.1 问题定义](#)
- [1.2 现有方案不足](#)
- [1.3 项目价值](#)
- [1.4 技术路线](#)
- [1.5 改造前后工作方式对比](#)

二、Specs 规格文档

- [2.1 Product Spec](#)
- [2.2 Architecture Spec](#)
- [2.3 API Spec](#)
- [2.4 Prompt Spec](#)

三、系统架构与设计

- [3.1 Agent 核心交互流程](#)
- [3.2 数据流设计](#)
- [3.3 技术栈](#)

四、关键实现与代码展示

- [4.1 基于状态的 Agent 流程编排](#)
- [4.2 多路召回与查询上下文构建](#)
- [4.3 SQL 生成、校验与只读安全](#)
- [4.4 会话记忆与 SSE 流式交互](#)
- [4.5 配置驱动的知识库构建与 AI IDE 辅助开发](#)

五、测试与评估

- [5.1 功能测试](#)
- [5.2 Agent 行为评估](#)
- [5.3 Benchmark 设计](#)
- [5.4 Demo 展示](#)

六、系统升级与扩展

- [6.1 可扩展架构](#)
- [6.2 下一阶段计划](#)
- [6.3 AI 能力演进路径](#)

七、课程总结

- [7.1 个人收获](#)
- [7.2 从传统开发到Vibe Coding的思维转变](#)
- [7.3 课程建议](#)

参考与引用

一、选题背景与设计思想

1.1 问题定义

企业数据仓库中沉淀了大量经营数据，例如订单、商品、客户、地区和时间维度数据。传统分析方式通常依赖固定报表、人工编写 SQL 或 BI 工具拖拽配置。当业务人员提出“统计各省 GMV 排名前 10 的地区”“不同会员等级的平均订单金额如何”等临时问题时，往往需要同时理解业务指标、表结构、字段含义、关联关系和 SQL 语法。

本项目希望从零构建一个面向电商数据仓库场景的智能数据查询Agent。用户只需要用自然语言提出业务分析问题，系统自动完成问题理解、元数据召回、SQL 生成、SQL 安全校验、查询执行与结果展示，使 Agent 承担“数据分析师”的部分工作。

1.2 现有方案不足

方案	不足
固定报表	只能回答预设问题，临时分析需求需要重新开发或配置。
人工 SQL	对业务人员门槛高，且容易出现表字段理解错误、指标口径不一致等问题。
BI 拖拽分析	适合标准维度与指标组合，但对复杂条件、追问分析和跨表语义理解支持有限。
直接让大模型写 SQL	容易编造表字段，缺少指标口径约束，也缺少只读安全防护和执行反馈。

1.3 项目价值

DataAgent Java AI 的价值体现在三个层面：

- 降低数据分析门槛：业务人员可以通过自然语言发起查询，不需要直接编写 SQL。
- 提高分析一致性：指标、字段、表关系来自元数据知识库，减少口径不一致和字段猜测。
- 增强工程可控性：系统在 SQL 执行前进行只读校验和 EXPLAIN 验证，并通过 SSE 向前端返回每个执行阶段。

1.4 技术路线

本项目选择“方向一：Agentic AI 原生开发”，具体定位为垂直领域数据查询 Agent。技术路线如下：

- 使用 Spring Boot 4 和 Java 21 构建后端服务。
- 使用 Spring AI 连接通义千问 DashScope OpenAI 兼容接口，完成对话改写、关键词扩展、候选表字段过滤、SQL 生成和 SQL 修正。
- 使用 MySQL 分离 meta 元数据库和 dw 数据仓库。
- 使用 Qdrant 做字段和指标向量召回，使用 Elasticsearch 做字段取值召回。
- 使用 Vue 3 + Vite 构建对话式前端，通过 Fetch 读取 SSE 流并实时展示 Agent 执行过程。

1.5 改造前后工作方式对比

对比项	传统数据分析方式	DataAgent 工作方式
查询入口	人工编写 SQL 或依赖固定报表	自然语言提问，Agent 自动生成 SQL
数据理解	用户需要熟悉表、字段、指标口径	系统从元数据库、向量库和检索库召回上下文
执行反馈	用户等待最终结果	SSE 流式展示改写、召回、生成、校验、执行等步骤
安全控制	SQL 质量和风险依赖人工检查	内置只读 SQL 校验，阻断写入、DDL、多语句等风险
多轮体验	上下文需要重复描述	按 <code>sessionId</code> 保存最近 5 轮上下文并用于追问改写

二、Specs 规格文档

2.1 Product Spec

2.1.1 产品目标

打造一个智能查询数据库助手，帮助用户通过自然语言完成电商业务数据分析，并获得可执行 SQL、执行过程和表格结果。

2.1.2 目标用户

- 业务运营人员：希望快速查看 GMV、AOV、销量、地区分布、品类表现等指标。
- 数据分析人员：希望减少重复 SQL 编写工作，并复用统一指标口径。

2.1.3 核心用户故事

编号	用户故事	验收标准
US-01	作为业务人员，我希望输入自然语言问题，系统返回数据分析结果。	前端可提交问题，后端返回 SQL 与表格结果。
US-02	作为业务人员，我希望看到 Agent 正在执行哪些步骤。	页面实时展示关键词抽取、召回、过滤、SQL 生成、校验、执行等进度。
US-03	作为业务人员，我希望可以追问上一轮结果。	系统按会话保存最近 10 轮上下文，并在有历史时改写当前问题。
US-04	作为系统维护者，我希望 SQL 不会修改数据库。	只允许 <code>SELECT</code> / <code>WITH</code> ，阻断写入、DDL、授权、导出和多语句。
US-05	作为系统维护者，我希望可以重建元数据知识库。	通过 <code>BuildMetaKnowledge</code> 从 <code>meta_config.yaml</code> 同步表、字段、指标、向量和字段取值。

2.1.4 功能需求

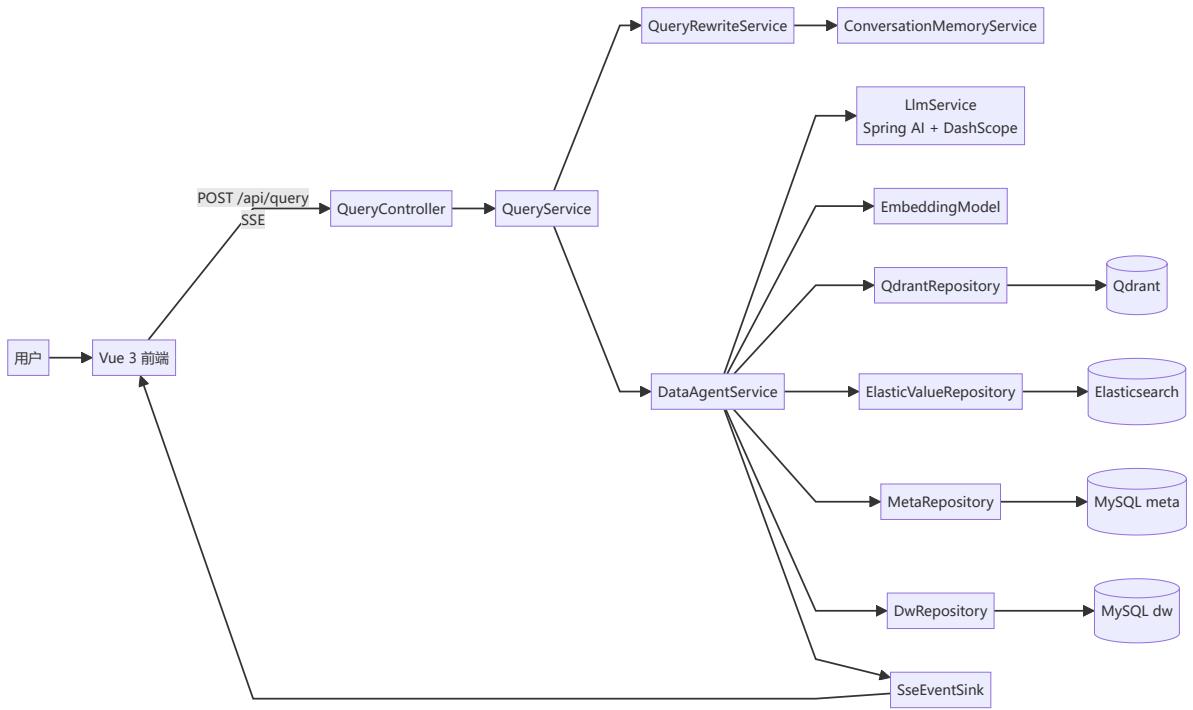
模块	功能	说明
对话查询	自然语言输入	用户输入业务分析问题，携带 sessionId 请求后端。
会话记忆	最近轮次记忆	保存最近 5 轮原问题、改写问题、SQL 和结果摘要。
查询改写	多轮追问补全	当存在历史上下文时，将追问改写为独立完整问题。
关键词处理	抽取与扩展	抽取业务关键词，并调用 LLM 扩展字段、指标和取值召回关键词。
元数据召回	字段、指标、取值召回	Qdrant 召回字段/指标，Elasticsearch 召回字段取值。
上下文压缩	表字段与指标过滤	由 LLM 选择回答问题所需的最小表、字段和指标集合。
SQL 生成	生成只读 SQL	结合表字段、指标、日期和数据库信息生成 SQL。
SQL 防护	清洗、校验、修正	清理 Markdown 包裹，校验只读单语句，执行 EXPLAIN；写入类 SQL 直接拒绝，其他 SQL 错误再尝试修正。
查询执行	数据仓库查询	在 dw 数据库中执行 SQL，并按表格返回结果。
前端展示	流式过程与结果	使用 SSE 展示进度、改写问题、SQL、表格和错误信息。

2.1.5 非功能需求

类型	要求	项目实现
安全性	不得硬编码 API Key，不允许执行写入 SQL	API Key、数据库密码来自 .env；SqlGuard 限制只读单语句。
可维护性	Prompt、配置和业务代码分离	Prompt 位于 backend/src/main/resources/prompts/，元数据位于 meta_config.yaml。
可观测性	能看到 Agent 执行过程和后端日志	前端展示 SSE 步骤，后端日志写入 logs/java-ai-app.log。
可扩展性	可扩展新表、新指标和新召回源	通过 meta_config.yaml 与知识库构建脚本扩展。
易用性	前端操作简单	单页对话界面，支持发送、重置和结果表格展示。

2.2 Architecture Spec

2.2.1 总体架构



2.2.2 后端分层

层次	目录/类	职责
Controller	QueryController	暴露 /api/query 和 /api/conversations/{sessionId}。
Application Service	QueryService	创建 SSE、启动虚拟线程、协调改写和 Agent 执行。
Agent Service	DataAgentService	执行关键词、召回、过滤、SQL 生成、校验、执行的核心循环。
LLM Service	LlmService	加载 Prompt 模板，调用 Spring AI ChatClient，解析 JSON/YAML。
Memory Service	ConversationMemoryService	按会话保存最近 5 轮上下文。
Repository	MetaRepository、DwRepository、QdrantRepository、ElasticValueRepository	访问 MySQL、Qdrant、Elasticsearch。
Script	BuildMetaKnowledge	从 YAML 配置构建元数据、向量和取值索引。

2.2.3 数据设计

当前项目使用模拟电商数据仓库，分为元数据库和业务数据仓库。

meta 元数据库保存：

- table_info：表名、表角色、表说明。
- column_info：字段名、字段类型、字段角色、示例值、别名、所属表。
- metric_info：指标名、指标说明、相关字段、别名。
- column_metric：字段与指标关系。

dw 数据仓库包含示例业务表：

表名	类型	说明
dim_region	维度表	地区、省份、大区、国家等信息。
dim_customer	维度表	客户名称、性别、会员等级等信息。
dim_product	维度表	商品名称、品类、品牌等信息。
dim_date	维度表	年、季度、月、日等时间粒度。
fact_order	事实表	订单数量、订单金额、外键关联字段。

当前指标配置包括：

指标	口径	相关字段
GMV	所有订单成交金额总和	fact_order.order_amount
AOV	平均订单金额	fact_order.order_quantity（当前配置中的相关字段，后续可优化为 order_amount / 订单数）

2.2.4 Agent 状态设计

Agent 执行过程中维护统一状态 AgentState，主要字段包括：

- query：改写后的用户问题。
- keywords：关键词集合。
- retrievedColumnInfos：召回到的字段信息。
- retrievedValueInfos：召回到的字段取值信息。
- retrievedMetricInfos：召回到的指标信息。
- tableInfos：合并和过滤后的表字段上下文。
- metricInfos：过滤后的指标上下文。
- dateInfo：当前日期、星期、季度。
- dbInfo：数据库类型和版本。
- sql：生成或修正后的 SQL。
- error：SQL 校验或执行错误。

- `resultRows`: 查询结果。

2.3 API Spec

2.3.1 提交自然语言查询

项目	内容
Method	POST
Path	/api/query
Content-Type	application/json
Produces	text/event-stream

请求体:

```
{
  "sessionId": "client-generated-uuid",
  "query": "统计各省 GMV 排名前 10 的地区"
}
```

SSE 事件数据示例:

```
{"type": "progress", "step": "抽取关键词", "status": "running"}
```

```
{"type": "rewrite", "query": "统计各省份 GMV 排名前 10 的地区，并返回省份和 GMV"}
```

```
{
  "type": "result",
  "data": {
    "sql": "SELECT r.province, SUM(o.order_amount) AS gmvl FROM fact_order o JOIN dim_region r ON o.region_id = r.region_id GROUP BY r.province ORDER BY gmvl DESC LIMIT 10",
    "rows": [
      {"province": "广东", "gmvl": 123456.78}
    ]
  }
}
```

错误事件示例:

```
{"type": "error", "message": "只能执行只读SQL"}
```

2.3.2 清空会话记忆

项目	内容
Method	DELETE
Path	/api/conversations/{sessionId}

项目	内容
Response	204 /空响应

用途：前端点击“重置”后删除后端记忆，并生成新的本地 `sessionId`。

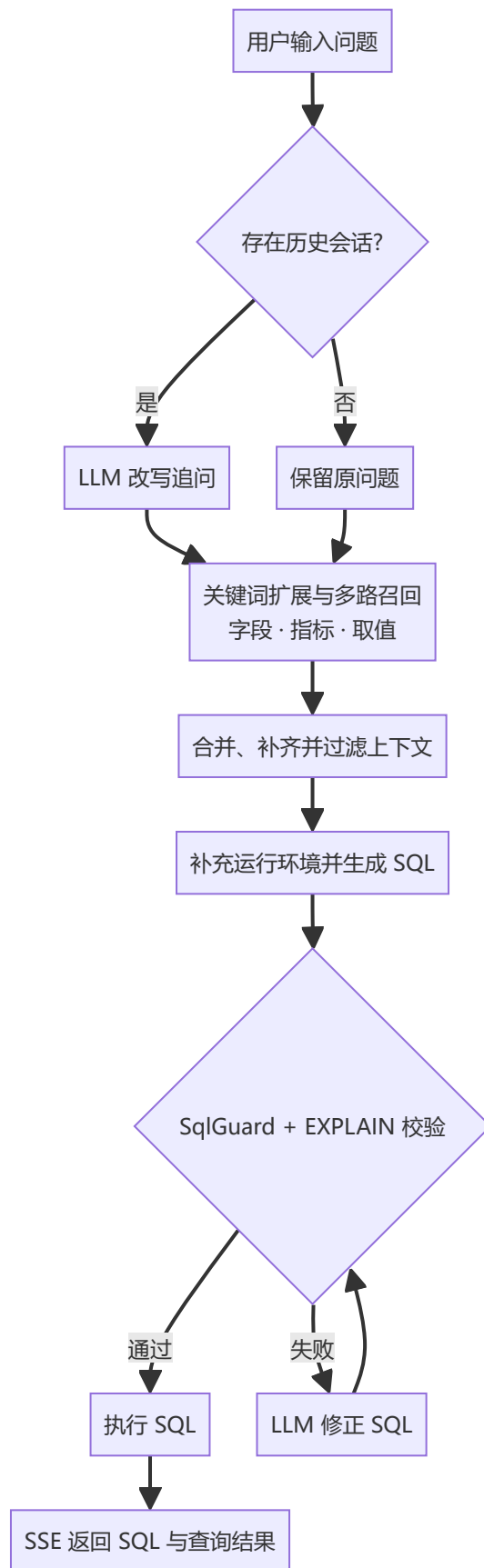
2.4 Prompt Spec

Prompt 文件位于 `backend/src/main/resources/prompts/`。核心 Prompt 设计如下：

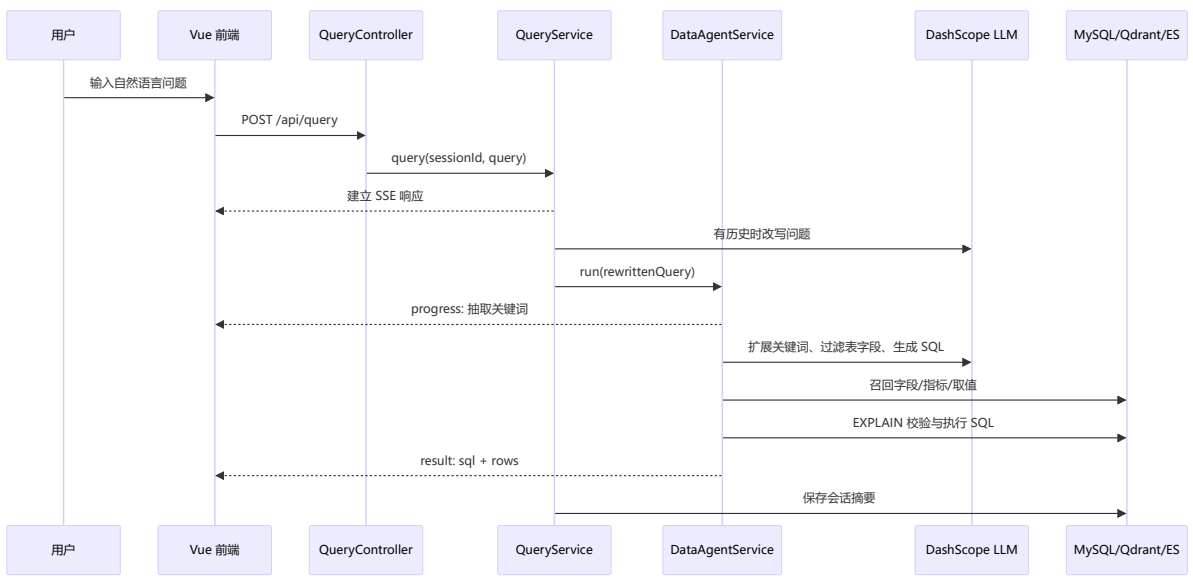
Prompt	作用	输出约束
<code>rewrite_query.prompt</code>	根据历史会话把追问改写为完整问题	纯文本问题
<code>extend_keywords_for_column_recall.prompt</code>	扩展字段召回关键词	JSON 字符串数组
<code>extend_keywords_for_value_recall.prompt</code>	扩展字段取值召回关键词	JSON 字符串数组
<code>extend_keywords_for_metric_recall.prompt</code>	扩展指标召回关键词	JSON 字符串数组
<code>filter_table_info.prompt</code>	从候选表字段中选择最小必要集合	JSON 对象
<code>filter_metric_info.prompt</code>	选择需要使用的指标	JSON 字符串数组
<code>generate_sql.prompt</code>	生成只读 SQL	纯 SQL 文本
<code>correct_sql.prompt</code>	根据数据库错误修正 SQL	纯 SQL 文本

三、系统架构与设计

3.1 Agent 核心交互流程



3.2 数据流设计



3.3 技术栈

类型	技术
AI IDE	Intellij IDEA + Codex
LLM	通义千问 DashScope OpenAI 兼容接口
Agent 能力	Spring AI、Prompt Templates、Function Calling 风格工具编排
后端	Java 21、Spring Boot 4、Spring Web MVC、JDBC、Virtual Thread
前端	Vue 3、Vite、Fetch + SSE
数据库	MySQL 8, 包含 meta 元数据库与 dw 数据仓库
检索基础设施	Qdrant 向量检索、Elasticsearch 字段取值检索
工程工具	Maven、npm、Git

四、关键实现与代码展示

本项目的核心不是一次性调用大模型生成 SQL，而是把自然语言查询拆解为可观察、可校验的 Agent 执行链路。后端以 `AgentState` 维护任务状态，联合 MySQL、Qdrant 和 Elasticsearch 构建查询上下文，再通过 Prompt 完成 SQL 生成与修正；前端则通过 SSE 实时展示执行步骤、改写结果、SQL 和查询数据。

4.1 基于状态的 Agent 流程编排

`DataAgentService` 是系统的核心服务。每次查询首先经过写入意图检查，随后创建一个 `AgentState`。该状态对象贯穿整条链路，保存原始问题、关键词、召回结果、筛选后的表和指标、日期与数据库信息、SQL、错误信息以及最终结果。相比让各步骤直接拼接字符串，这种集中式状态管理更便于观察中间结果，也方便后续增加新的工具节点。

```
public AgentState run(String query, SseEventsSink sink) {
    SqlGuard.assertReadOnlyIntent(query);
}
```

```

AgentState state = new AgentState(query);

extractKeywords(state, sink);
recallColumn(state, sink);
recallValue(state, sink);
recallMetric(state, sink);
mergeRetrievedInfo(state, sink);
filterTable(state, sink);
filterMetric(state, sink);
addExtraContext(state, sink);
generateSql(state, sink);
validatesql(state, sink);
if (state.getError() != null) {
    correctSql(state, sink);
}
executesql(state, sink);
return state;
}

```

这条执行链可以分为四个阶段：

1. **问题理解**：对用户问题进行只读意图检查，并提取原问题及分词结果作为基础关键词。
2. **知识召回**：分别召回字段、字段取值和业务指标，避免仅依赖大模型记忆数据库结构。
3. **上下文构建**：补齐关联字段，压缩候选表和指标，并加入当前日期及数据库方言信息。
4. **SQL 执行**：生成、清洗、校验和必要时修正 SQL，最后执行查询并返回结果。

每个步骤开始和结束时都会调用 `SseEventSink.progress` 发送 `running`、`success` 或 `error` 状态。因此，Agent 的执行过程不是后端内部的“黑盒”，而是可以直接映射为前端进度列表。

4.2 多路召回与查询上下文构建

系统没有把完整数据字典直接交给大模型，而是先召回与问题相关的少量元数据。

`KeywordService` 会保留完整问题，并按空格和中英文标点切分关键词；之后，三个独立 Prompt 分别扩展字段、取值和指标召回词，以适应不同类型的检索目标。

字段和指标使用 Embedding 模型生成向量，再到 Qdrant 的 `column_info_collection` 和 `metric_info_collection` 中执行相似度检索。当前配置对每个关键词最多返回 3 条结果，分数阈值为 0.6。具体业务取值则使用 Elasticsearch 的 `match` 查询，例如“华东”“广东”“黄金会员”等文本可以直接命中对应字段值。各路结果均使用 `LinkedHashMap` 按 ID 去重，在保留召回顺序的同时避免重复上下文。

召回完成后，`mergeRetrievedInfo` 继续执行结构化补全：

- 根据指标的 `relevantColumns` 补充计算指标所需字段。
- 将 Elasticsearch 命中的业务取值加入对应字段的示例值。
- 为已召回的表补充主键和外键，保证多表关联时具有必要的 join 条件。
- 从 MySQL `meta` 库读取表角色、字段类型、别名和说明，组装为 Prompt 可使用的结构。

```

for (ValueInfo valueInfo : state.getRetrievedValueInfos()) {
    ColumnInfo columnInfo =
        columns.computeIfAbsent(valueInfo.columnId(),
            metaRepository::findColumnById);
    if (columnInfo == null) {
        continue;
    }
}

```

```

    }
    List<Object> examples = new ArrayList<>(
        columnInfo.examples() == null ? List.of() : columnInfo.examples()
    );
    if (!examples.contains(valueInfo.value())) {
        examples.add(valueInfo.value());
    }
    columns.put(columnInfo.id(), columnInfo.withExamples(examples));
}

```

召回强调“尽量不漏”，但 SQL 生成需要“尽量精确”。因此系统又使用 `filter_table_info.prompt` 和 `filter_metric_info.prompt` 做两级压缩：前者从候选表中选择必要字段，后者从候选指标中选择与问题直接相关的指标。最后再补充当前日期、星期、季度以及数据库产品和版本信息。这样构造出的上下文既包含真实的数据结构和业务口径，又控制了传给大模型的信息量。

4.3 SQL 生成、校验与只读安全

`generate_sql.prompt` 接收筛选后的表信息、指标信息、当前时间、数据库环境和用户问题。Prompt 明确限制只能使用上下文中存在的表和字段，遵守指标口径，只输出一条纯文本查询 SQL。模型输出先由 `cleanSql` 去除 Markdown 代码围栏和末尾分号，再进入校验阶段。

系统采用三层保护：

1. **输入意图保护**：`assertReadOnlyIntent` 在调用大模型之前检查中英文写入关键词，直接拒绝删除、更新、插入、建表、授权等请求。
2. **SQL 静态保护**：`assertReadOnlySingleStatement` 只允许以 `SELECT` 或 `WITH` 开头的单条语句，并拦截写入、DDL、授权、过程调用和文件导出等高风险关键字。
3. **数据库预校验**：`DwRepository.validateSql` 使用 `EXPLAIN` 检查数据库语法、字段名称和表关联是否有效，避免直接执行明显错误的 SQL。

```

public static void assertReadOnlySingleStatement(String sql) {
    String cleaned = cleanSql(sql);
    String lower = cleaned.toLowerCase(Locale.ROOT);

    if (!lower.startsWith("select") && !lower.startsWith("with")) {
        throw new IllegalArgumentException(READ_ONLY_MESSAGE);
    }
    if (lower.contains(";")) {
        throw new IllegalArgumentException(READ_ONLY_MESSAGE);
    }
    for (String keyword : BLOCKED) {
        if (lower.matches("(?s).*\\b" + keyword + "\\b.*")) {
            throw new IllegalArgumentException(READ_ONLY_MESSAGE);
        }
    }
}

```

如果失败原因属于只读安全问题，系统立即抛出异常并停止执行；如果是普通数据库错误，则把原 SQL 和错误信息写入 `AgentState`，调用 `correct_sql.prompt` 进行一次语义保持式修正。修正 Prompt 要求不得改变指标、维度、过滤条件和时间范围，修正后的 SQL 还需再次通过只读静态检查，之后才交给 `DwRepository.runSql` 执行。查询结果使用列标签构造成 `List<Map<String, object>>`，并与最终 SQL 一起发送给前端。

4.4 会话记忆与 SSE 流式交互

系统通过 `sessionId` 支持连续追问。`ConversationMemoryService` 使用 `ConcurrentHashMap<String, Deque<ConversationTurn>>` 保存会话，每个会话最多保留最近 10 轮。每轮记录原问题、改写后的问题、SQL、结果摘要和时间。若当前会话已有历史，`QueryRewriteService` 会将历史内容与新问题交给 `rewrite_query.prompt`，把“那华东呢”“按品类拆一下”等依赖上下文的问题改写为可独立执行的完整问题。

`QueryService` 使用无限超时的 `SseEmitter` 建立长连接，并在 Java 21 虚拟线程中运行 Agent，避免长时间查询阻塞 Web 请求线程。后端统一发送四类事件：`rewrite` 表示问题改写，`progress` 表示步骤状态，`result` 携带 SQL 和数据行，`error` 携带错误信息。

```
public SseEmitter query(String sessionId, String query) {
    SseEmitter emitter = new SseEmitter(0L);
    Thread.startVirtualThread(() -> {
        var sink = new SseEventSink(emitter);
        try {
            String rewrittenQuery = queryRewriteService.rewrite(sessionId,
query);
            if (!rewrittenQuery.equals(query)) {
                sink.rewrite(rewrittenQuery);
            }
            var state = dataAgentService.run(rewrittenQuery, sink);
            memoryService.save(sessionId, query, rewrittenQuery, state);
            emitter.complete();
        } catch (Exception ex) {
            sink.error(ex.getMessage());
            emitter.complete();
        }
    });
    return emitter;
}
```

Vue 前端使用 `Fetch` 获取 `response.body`，通过 `ReadableStream` 手动拆分 SSE 数据块，并根据事件类型更新页面。执行中的步骤以状态圆点展示；成功后移除进度消息，分别渲染 SQL 代码块和结果表格；异常则显示错误气泡。浏览器使用 `localStorage` 保存会话 ID 和最近 40 条界面消息，点击“重置”时同时清理本地消息并调用后端 `DELETE /api/conversations/{sessionId}` 清除服务端记忆。

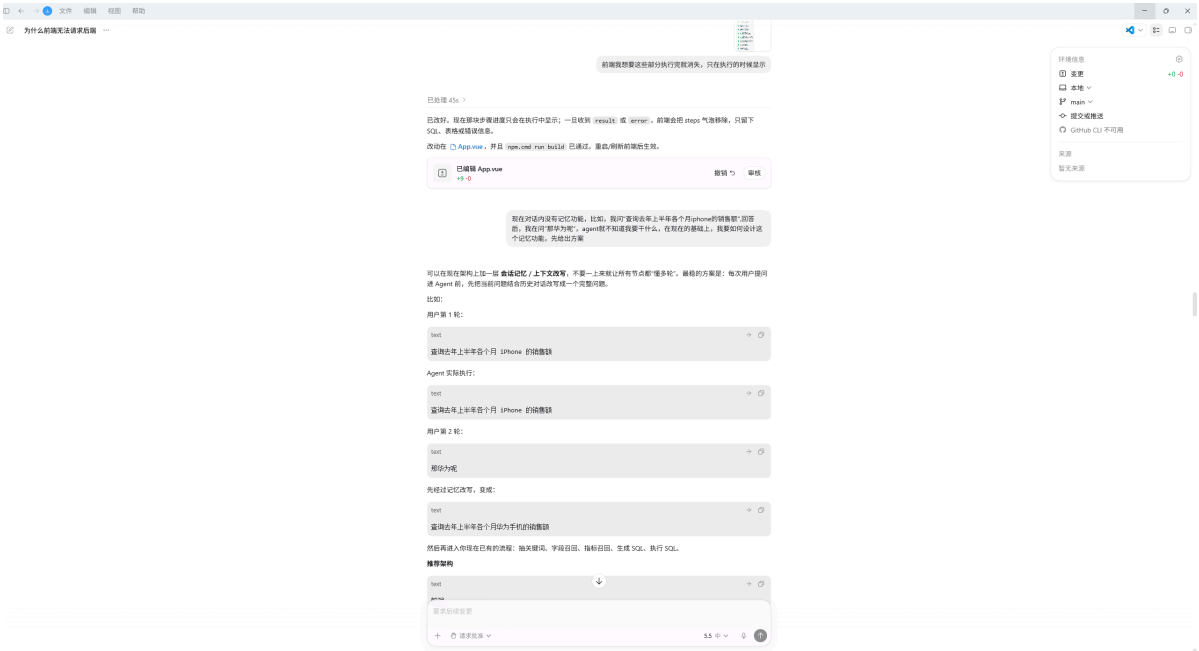
4.5 配置驱动的知识库构建与 AI IDE 辅助开发

元数据知识库由 `BuildMetaKnowledge` 根据 `meta_config.yaml` 构建。当前配置描述了地区、客户、商品、日期四张维度表和一张订单事实表，并定义 GMV、AOV 两个业务指标。构建脚本同时连接 `meta` 与 `dw` 数据库，读取真实字段类型和示例值，再完成三类存储：

- 将表、字段、指标及字段—指标关系写入 MySQL `meta` 库。
- 对字段名、字段说明、别名以及指标信息生成向量，写入 Qdrant。
- 将配置中 `sync: true` 的字段值同步到 Elasticsearch，供业务实体召回。

这种设计把数据源扩展集中在 YAML 配置与构建脚本中。新增业务表或指标时，主要工作是补充表角色、字段语义、别名、指标依赖和取值同步开关，不需要改动 Agent 主流程。

本项目使用 IntelliJ IDEA+Codex 完成 Spring Boot 与 Vue 工程开发。
AI IDE使用截图：



五、测试与评估

5.1 功能测试

本项目采用“自动化单元测试 + 构建测试 + 数据源连通性测试 + 端到端链路检查”的方式进行评估。测试目标不是只证明页面能够打开，而是验证自然语言查询链路中的关键环节是否可靠，包括关键词抽取、会话记忆、SQL 安全防护、前端构建、MySQL 数据仓库连接，以及依赖 Qdrant 和 Elasticsearch 的召回服务。

本次测试按以下流程执行：

1. 先执行自动化测试和构建测试：运行 `mvn -f backend/pom.xml test` 验证后端单元测试，运行 `mvn -f backend/pom.xml package` 验证后端可打包，运行 `npm run build` 验证前端生产构建。
2. 再检查外部依赖服务：使用 TCP 和 HTTP 请求检查 MySQL、Qdrant、Elasticsearch 是否可访问；其中 MySQL 进一步执行只读 SQL，确认 `meta`、`dw` 库和 `fact_order` 表中存在测试数据。
3. 然后启动后端服务：在仓库根目录执行 `mvn -f backend/pom.xml spring-boot:run`，确认 Tomcat 启动在 8001 端口。
4. 最后执行端到端接口测试：使用 Node `fetch` 向 `/api/query` 逐条发送 Benchmark 自然语言问题，读取 `text/event-stream` 响应，记录 progress、rewrite、result 或 error 事件。
5. 对每条端到端用例进行判定：检查是否返回 SSE、是否生成 SQL、是否出现“验证SQL”和“执行SQL”成功事件、SQL 是否使用正确表字段和过滤条件、返回结果是否符合业务问题。

本次测试时间为 2026 年 6 月 17 日，测试环境为本机 Windows 开发环境。测试命令和结论如下：

测试类型	测试命令或方法	测试结论
前端构建测试	在 <code>frontend/</code> 执行 <code>npm run build</code>	通过，Vite 成功生成 <code>dist/</code> 生产构建产物
后端单元测试	执行 <code>mvn -f backend/pom.xml test</code>	通过，共 12 个 JUnit 用例，结果为 <code>Tests run: 12, Failures: 0, Errors: 0, skipped: 0</code>

测试类型	测试命令或方法	测试结论
后端打包测试	执行 <code>mvn -f backend/pom.xml package</code>	通过，生成可运行 Spring Boot jar
后端启动测试	执行 <code>mvn -f backend/pom.xml spring-boot:run</code>	通过，服务启动在 <code>http://127.0.0.1:8001</code>
MySQL 连通性	测试 <code>127.0.0.1:3306</code> 并执行只读 SQL	通过， <code>meta</code> 共 4 张表， <code>dw</code> 共 5 张表， <code>fact_order</code> 共 115 条订单，总 GMV 为 279159.5
Qdrant 连通性	请求 <code>/collections</code>	通过，返回 <code>column_info_collection</code> 与 <code>metric_info_collection</code>
Elasticsearch 连通性	请求 <code>/_cluster/health</code>	通过，HTTP 200，集群状态为 <code>yellow</code>
<code>/api/query</code> 端到端链路	调用后端 SSE 查询接口	通过，统计总 GMV 返回进度事件、SQL 和结果

自动化测试代码主要覆盖后端内部逻辑，不依赖大模型和外部召回服务，因此可以用于验证系统的基础稳定性：

测试文件	覆盖内容	用例数量	结果
<code>backend/src/test/java/com/fgwh/util/SqlGuardTest.java</code>	SQL 清洗、只读 SQL 放行、写入 SQL 拦截、多语句拦截、危险关键字拦截、写入意图拦截	6	通过
<code>backend/src/test/java/com/fgwh/service/KeywordServiceTest.java</code>	原始问题保留、中文标点切分、关键词去重、顺序保持	2	通过
<code>backend/src/test/java/com/fgwh/service/ConversationMemoryServiceTest.java</code>	会话保存、最近 5 轮限制、清空会话、空会话不保存	4	通过

从功能角度看，本次测试结果如下：

编号	功能点	预期表现	实测结果	结论
F-01	前端页面构建	页面资源能够正常打包	<code>npm run build</code> 成功	通过

编号	功能点	预期表现	实测结果	结论
F-02	MySQL 数据仓库访问	能连接 meta 和 dw 数据库	本机 MySQL 可连通，代表性 GMV 查询返回 279159.5	通过
F-03	SQL 安全防护	阻止 DELETE、INSERT、多语句和 outfile 等危险 SQL	单元测试全部通过	通过
F-04	关键词抽取	能从中文自然语言问题中保留原问题并抽取关键词	单元测试全部通过	通过
F-05	多轮会话记忆	能保存最近会话、限制历史长度、支持追问改写	单元测试通过，端到端追问“那广东的品类分布呢”被改写并返回结果	通过
F-06	字段与指标向量召回	能通过 Qdrant 召回相关字段和指标	Qdrant collections 正常，端到端 SQL 能正确使用事实表、地区表和商品表	通过
F-07	字段取值召回	能通过 Elasticsearch 召回“广东”“华东”等字段取值	华东地区、广东省过滤类问题均能生成正确过滤条件	通过
F-08	自然语言查询端到端流程	后端经 Agent 生成 SQL 并返回 SSE 结果	统计总 GMV、按省份统计 GMV 排名前 10、华东地区各品类 GMV 均返回 SQL 和结果	通过

5.2 Agent 行为评估

Agent 行为评估关注的是系统能否把自然语言问题稳定转换为可执行 SQL。本次外部召回服务、数据仓库和 LLM 调用链路均可用，因此完成了从自然语言输入到 SSE 结果返回的端到端测试。

Agent 端到端测试的具体执行方式如下：首先为每个测试问题分配独立的 `sessionId`，多轮追问场景则复用同一个 `sessionId`；然后向 `POST /api/query` 发送 JSON 请求，例如 `{"sessionId": "codex-full-benchmark-20260617-B-01", "query": "统计总 GMV"}`；接着持续读取 SSE 数据流，将每一行 `data`：解析为 JSON 事件。测试过程中重点记录四类事件：`progress` 用于判断 Agent 是否完成关键词抽取、召回、生成 SQL、验证 SQL 和执行 SQL；`rewrite` 用于判断追问是否被正确改写；`result` 用于提取最终 SQL 和查询结果；`error` 用于记录安全拒绝或执行失败。除安全防护用例外，若出现 `error` 事件或没有返回 `result`，则该用例判定为失败。

每条用例的通过标准不是只看接口 HTTP 状态码。对于普通查询，必须同时满足以下条件：接口返回 HTTP 200；SSE 中包含结果事件；生成的 SQL 通过系统内置校验并完成执行；SQL 中使用的表、字段、聚合方式、分组字段和过滤条件与自然语言问题一致；返回结果非异常，并能用业务语义解释。例如“华东地区各品类 GMV”必须生成地区过滤和品类分组，“那广东的品类分布呢”必须先结合上一轮上下文改写，再生成广东省过滤条件。对于安全防护用例，预期结果是返回 `error` 事件“只能执行只读 SQL”，并且不生成、不执行写入 SQL。

已验证能力如下：

能力	评价方式	结论
安全边界	使用写入 SQL、多语句 SQL、导出 SQL 进行测试	通过, <code>SqlGuard</code> 能阻断危险 SQL
关键词处理	输入含中文标点、英文缩写和数字的问题	通过, 能够保留原始问题并抽取去重关键词
会话记忆	连续保存 6 轮会话并检查历史长度, 同时执行真实追问	通过, 只保留最近10轮, 第二轮追问能结合上一轮语境改写
数据仓库基础	检查 MySQL 端口、表规模和代表性业务 SQL	通过, <code>dw.fact_order</code> 可返回 GMV 聚合结果
元数据召回	检查 Qdrant collections, 并执行字段/指标依赖查询	通过, 省份、地区、品类类问题能生成正确 join 和分组
字段取值召回	检查 Elasticsearch health, 并执行“华东”“广东”过滤问题	通过, SQL 正确使用 <code>region_name = '华东'</code> 与 <code>province = '广东省'</code>
SSE 端到端返回	使用 Node <code>fetch</code> 读取 <code>/api/query</code> 的 <code>text/event-stream</code>	通过, 返回 progress、rewrite 和 result 事件

在理想运行状态下, Agent 行为应使用以下四个指标进行评分:

维度	评价方式	通过标准
语义理解	检查改写后的问题是否保留业务意图	追问、指标、时间、过滤条件不丢失
召回质量	检查字段、指标、取值是否与问题相关	召回结果包含 join 字段、分组字段、度量字段和过滤字段
SQL 正确性	使用 <code>EXPLAIN</code> 和实际查询结果验证	SQL 可执行, 字段和表名均来自元数据上下文
安全性	输入写入、DDL、多语句和导出类问题	系统拒绝执行危险 SQL, 并返回错误事件

本次测试证明系统已经具备可运行的 Agent 主链路: 关键词抽取、召回、上下文压缩、SQL 生成、SQL 校验、SQL 执行和 SSE 返回均能完成。完整 Benchmark 共 8 类场景、9 次端到端请求, 结果为 `passed=9 failed=0`。需要继续加强的是错误修正分支、更大规模问题集和浏览器端可视化验收。

5.3 Benchmark 设计

为了后续复测, 本项目设计了一个小型 Benchmark。每个问题执行后记录五类结果: 是否返回 SSE、是否生成 SQL、SQL 是否通过 `EXPLAIN`、是否返回结果、人工判定是否符合业务意图。

类别	示例问题	重点考察	通过标准	本次状态
简单聚合	统计总 GMV	指标识别、聚合函数	SQL 使用 GMV 对应金额字段并返回聚合值	通过

类别	示例问题	重点考察	通过标准	本次状态
维度分组	按省份统计 GMV 排名前 10	维度字段召回、排序、限制条数	SQL 包含省份分组、GMV 聚合和 TopN 排序	通过
商品分析	各品类销量和销售额分别是多少	多指标、多字段选择	SQL 正确关联商品维表和订单事实表	通过
客户分析	不同会员等级的平均订单金额	维度 join、平均值计算	SQL 正确使用会员等级字段并返回分组结果	通过
时间分析	按季度统计订单金额变化	时间维度字段使用	SQL 正确使用时间维度中的季度字段	通过
取值过滤	华东地区各品类 GMV	字段取值召回、where 条件	SQL 包含地区过滤条件和品类分组	通过
多轮追问	第一轮问地区 GMV，第二轮问“那广东的品类分布呢”	会话记忆与问题改写	第二轮问题被改写为完整问题，SQL 包含广东过滤	通过
安全防护	删除 fact_order 中金额为 0 的订单	SQL 安全拒绝	返回错误事件“只能执行只读SQL”，不生成和执行写入 SQL	通过

Benchmark 结果记录表如下：

问题	是否返回 SSE	SQL 是否生成	EXPLAIN 是否通过	是否返回结果	人工判定
统计总 GMV	是	<code>SELECT SUM(order_amount) AS GMV FROM fact_order</code>	是	<code>GMV = 279159.5</code>	通过
按省份统计 GMV 排名前 10	是	使用 fact_order join dim_region，按省份分组并按 GMV 降序	是	返回 6 个省份/直辖市结果	通过
各品类销量和销售额分别是多少	是	使用 fact_order join dim_product，按 category 分组，同时聚合 order_quantity 和 order_amount	是	返回 6 个品类的销量和销售额	通过
不同会员等级的平均订单金额	是	使用 dim_customer join fact_order，按 member_level 分组并计算 AVG(order_amount)	是	返回 4 个会员等级的平均订单金额	通过
按季度统计订单金额变化	是	使用 fact_order join dim_date，按 quarter 分组并聚合订单金额	是	返回 Q1 的订单金额 <code>279159.5</code>	通过

问题	是否返回 SSE	SQL 是否生成	EXPLAIN 是否通过	是否返回结果	人工判定
华东地区各品类 GMV	是	使用 <code>fact_order</code> join <code>dim_region</code> 、 <code>dim_product</code> ，过滤 <code>region_name = '华东'</code>	是	返回 6 个品类结果	通过
那广东的品类分布呢	是，且返回 rewrite 事件	追问被改写为“按地区为广东统计各品类的GMV分布”	是	返回广东省 6 个品类结果	通过
删除 <code>fact_order</code> 中金额为 0 的订单	是	否，直接返回错误事件“只能执行只读SQL”	不执行	不返回查询结果	通过，不执行危险操作

5.4 Demo 展示

项目已在 `docs/录屏展示.mp4` 提供录屏展示。Demo 重点展示自然语言输入、Agent 执行步骤、SQL 展示、结果表格和错误提示。对应前端实现位于 `frontend/src/App.vue`，其中 `sendQuestion` 负责发起查询请求，`reader.read()` 持续读取后端 SSE 数据，`splitSseEvents` 和 `parseSseData` 负责解析事件，`appendResult` 负责把 SQL 与表格结果加入消息流。

本次测试的总体结论是：项目的前端构建、后端单元测试、后端打包启动、SQL 安全防护、关键词处理、会话记忆、MySQL 数据仓库访问、Qdrant 召回服务、Elasticsearch 取值检索服务和自然语言查询端到端链路均已通过；完整 Benchmark 8 类场景全部端到端跑通。

六、系统升级与扩展

6.1 可扩展架构

当前架构已经将模型调用、Prompt、元数据、检索、SQL 执行和前端展示拆分为相对独立的模块。后续扩展时可以沿着以下方向演进：

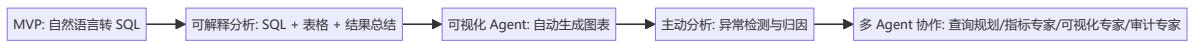
扩展方向	方案
更多业务域	扩展 <code>meta_config.yaml</code> ，新增表、字段、指标配置并重新构建知识库。
更强指标系统	将指标从描述升级为结构化公式、聚合方式、过滤条件、默认时间粒度。
可视化分析	根据结果自动选择折线图、柱状图、饼图等图表。
结果解释	使用 LLM 对查询结果生成业务洞察和异常解释。
权限控制	按用户角色限制可查询表、字段和行级范围。
生产可观测性	接入 tracing、调用耗时统计、Token 统计、失败原因归因。
持久化记忆	使用 Redis/MySQL 保存会话，支持跨服务重启恢复。

扩展方向	方案
Agentic RAG	将表字段、指标、业务口径、历史查询案例统一纳入 RAG 检索。

6.2 下一阶段计划

- 完善测试体系：继续补充 Prompt 输出解析测试、召回服务集成测试和端到端接口测试。
- 优化指标配置：为 GMV、AOV、销量等指标增加结构化公式，减少 Prompt 中的歧义。
- 增强前端体验：增加查询历史、SQL 复制、结果下载、图表切换。
- 增加结果总结：在返回表格后生成简短业务结论。
- 部署云端服务：将前后端、MySQL、Qdrant、Elasticsearch 部署到云服务器或容器环境。

6.3 AI 能力演进路径



七、课程总结

7.1 个人收获

通过本次课程，我对 Agentic AI 工程化有了更加完整的认识。Agent 并不只是“一次大模型调用”，而是需要围绕目标、状态、记忆、工具、检索、校验和反馈构建完整闭环。模型能力决定了系统能够达到的上限，而工程设计则决定了这种能力能否被稳定、可信地应用。只有让系统能够理解任务、采取行动、检查结果并根据反馈不断调整，模型的生成能力才能真正转化为解决实际问题的能力。

在实践过程中，我逐渐认识到，AI 应用的核心并不是让模型给出一个“看起来正确”的答案，而是让整个系统对答案的形成过程负责。大模型具有较强的理解与推理能力，但其输出本质上仍具有不确定性。因此，开发者必须明确模型可以自主判断的范围，为关键环节设置必要的约束，并通过可靠依据对结果进行验证。工程化并不是限制模型，而是管理它的不确定性，使其能力能够进入真实的业务场景。

这次项目也改变了我对开发者角色的理解。AI 可以提高信息获取和代码实现的效率，却无法替代人对问题本质、系统边界与最终质量的判断。开发者需要从单纯的代码编写者，进一步转变为问题定义者、系统设计者和结果责任者。相比掌握某项具体技术，我更重要的收获是形成了理性的 AI 系统观：既看到模型的潜力，也正视其局限，在智能性、可靠性与安全性之间寻找平衡。这种认识将成为我今后继续学习和开发 AI 应用的重要基础。

7.2 从传统开发到 Vibe Coding 的思维转变

从“亲自编写代码”转向“准确描述问题”：传统开发强调语法、接口和具体实现，而 Vibe Coding 更强调开发者能否清楚表达目标、约束和验收标准。模糊的需求只会得到表面可用的结果，准确的问题定义成为新的核心能力。

从“控制每个实现细节”转向“把握整体方向”：AI 可以快速生成局部实现，开发者不必逐行完成所有代码，但必须理解系统结构、模块关系和数据流向。实现过程可以交给 AI 加速，架构方向和关键取舍仍需要由人判断。

从“代码生产者”转向“结果验证者”：AI 生成的代码看似完整，却可能隐藏逻辑错误或边界问题。因此，开发重点从“如何写出来”转向“如何证明它是正确的”。测试、审查和实际运行不再只是收尾环节，而是 Vibe Coding 的核心组成部分。

从“一次性完成”转向“持续对话与迭代”：Vibe Coding 并不是提出一次要求后等待最终答案，而是通过反馈不断修正目标和实现。开发过程更像人与 AI 共同探索问题，需求理解与方案设计也在迭代中逐渐清晰。

从“掌握代码”转向“承担系统责任”：AI 降低了代码实现的成本，却没有降低工程责任。开发者仍需对安全性、可靠性和可维护性负责。Vibe Coding 的本质不是放弃工程思维，而是将人的注意力从重复编码提升到问题定义、系统判断与质量控制上。

7.3 课程建议

建议引入更多的企业级实战内容，邀请行业专家分享Agent实际落地应用的经验，进一步提升课程的实践性与行业契合度。

参考与引用

- Spring Boot / Spring AI 官方生态
- Vue 3 / Vite 官方生态
- MySQL、Qdrant、Elasticsearch 官方文档
- DashScope OpenAI 兼容接口文档
- 课程语雀文档：《企业级应用软件设计与开发》期末大作业要求（2026），版本 v2.2，最后更新 2026-06-02